



UNIVERSITY
of
TECHNOLOGY,
MAURITIUS

School of Innovative Technologies and Engineering (SITE)

BSc (Hons) Software Engineering

Object Oriented Programming Technical Report Java Year_Calendar_Application

Name: RANAIVOMAMPIONONA FENOHASINA MIHANTA Johane

Student ID: 2303_26116 **COHORT:** BSE23AFT(1)

Introduction:

The Year_Calendar_Application is a Java-based software designed to provide users with a graphical interface for viewing and navigating through a calendar displaying all twelve months of a given year.

Calendar Interface Description:

The calendar interface in the Year_Calendar_Application is designed to resemble the familiar layout of a phone's calendar application, offering users a seamless and intuitive experience similar to what they would encounter on their mobile devices.

Month Panels:

- Similar to a phone's calendar app, each month is presented as a separate panel within a grid layout.
- The month panels display the name of the month prominently at the top, with a font style and size that are consistent with mobile interfaces.

Current Day Highlight:

The Java Calendar Application incorporates a feature that highlights the current day within the displayed calendar. This functionality enhances the user experience by providing a visual cue to easily identify the present date.

Display Current year:

Another notable feature of the Java Calendar Application is its ability to display the calendar for the current year. This feature provides users with immediate access to an overview of all twelve months of the ongoing year, facilitating long-term planning and scheduling.

Switch to Any Year and Display by Month or Year:

One of the key functionalities of the Java Calendar Application is its flexibility in allowing users to switch to any year and choose between viewing the calendar by month or year. In the month view mode, users can focus on a specific month within the selected year, providing detailed insights into individual months and facilitating planning for specific events or activities.

Year View:

Month view:

Implementation Details:

This Java code is for a simple year calendar application with Swing GUI components. With 2 classes.

1-Import Java:

```
packages Swing GUI: - import java.awt.*;
                    -import javax.swing.*;

package java.time: import java.time.LocalDate;

package java.awt.event:- import java.awt.event.MouseEvent;
                      - import java.awt.event.MouseListener;
```

sources: <https://docs.oracle.com/javase/8/docs/api/java/time/package-summary.html>
<https://docs.oracle.com/javase/8/docs/api/java/awt/event/package-summary.html>

2-Swing-based GUI application :

Class 1: public class Year_Calendar_Application extends JFrame {

Class 2: public class calendar extends JFrame {

3-Private member variables :

Class Year_Calendar_Application:

```
    private Container container;
    private LocalDate currentDate;
    private JPanel monthGridPanel;
```

Class calendar:

```
    private JLabel monthLabel, yearLabel;
    private JButton prevMonthButton, nextMonthButton, prevYearButton, nextYearButton;
    private JPanel calendarPanel;
    private JButton[][] dayButtons;
    private int month, year;
    private Container container;
    private LocalDate currentDate;
```

4-constructor:

Java

```
public Year_Calendar_Application() {
    super("Calendar");
```

```

        container = getContentPane();
        container.setLayout(new BorderLayout());

public Calendar() {
    super("Calendar");
    container = getContentPane();
    container.setLayout(new BorderLayout());
}

```

5-Header Panel:

Class Year Calendar Application:

Instead of putting header panel , i use JLabel to have more flexibility

Java

```

// Create and add the label for the year 2024 at the top
JLabel yearLabel = new JLabel("2024");
yearLabel.setBorder(BorderFactory.createEmptyBorder(0,
    20, 0, 0)); // Set border container.add(yearLabel,
    BorderLayout.NORTH);
yearLabel.setFont(new Font("Serif", Font.PLAIN, 40)); // Set
font for the year label

```

Class calendar:

- For monthLabel, the text is set based on the current value of the month variable. It retrieves the corresponding month name from the array of month names and sets it as the label's text.
- For yearLabel, the text is set to the value of the year variable, converted to a string using String.valueOf(year).
- Icons are for navigation button to switch between next/previous year/month

Java

```

// Header Panel
JPanel headerPanel = new JPanel(); // Create header panel
monthLabel = new JLabel(" ", JLabel.CENTER); // Label for

```

```

month
yearLabel = new JLabel(" ", JLabel.CENTER); // Label for year

// Create custom icons for navigation buttons
Icon leftArrowIcon = new
ImageIcon("C:\\Demo\\right_row.png");// path of the
image Icon rightArrowIcon = new
ImageIcon("C:\\Demo\\left_row.png");

// Create buttons with custom icons for navigation
prevMonthButton = createIconButton(leftArrowIcon, "previous
month");
nextMonthButton = createIconButton(rightArrowIcon, "Next
month");
prevYearButton = createIconButton(leftArrowIcon, "previous
Year");
nextYearButton = createIconButton(rightArrowIcon, "next
Year");

// Method to update the calendar view with the current month and
year
private void updateCalendar() {
    // Set text for month and year labels
    monthLabel.setText(new String[]{"January", "February",
"March", "April", "May", "June", "July",
    "August", "September", "October", "November",
    "December"}[month]);
    yearLabel.setText(String.valueOf(year)); //static method of
String Class
    ...
}

```

6 -currentDate = LocalDate.now();

serves to obtain the current date from the system clock in the default time-zone of the system. It assigns this current date to the variable currentDate, allowing the application to work with the current date for various purposes such as displaying the current month or highlighting the current day in the calendar interface.

(<https://www.geeksforgeeks.org/localdate-now-method-in-java-with-examples/>)

7- The MonthPanelClickListener class is a listener for mouse click events on panels

representing each month in the calendar application. When a user clicks on one of these panels, it opens a new window displaying a detailed calendar for the selected month.

Java

```
public MonthPanelClickListener(int month) {
    this.month = month;
}

// Open the calendar when a grid is clicked
@Override
public void mouseClicked(MouseEvent e) {
    Calendar calendar = new Calendar(); // Create calendar window
    calendar.setVisible(true); // Set calendar window as visible
}

// Other mouse event methods
@Override
public void mousePressed(MouseEvent e) {}
@Override
public void mouseReleased(MouseEvent e) {}
@Override
public void mouseEntered(MouseEvent e) {}
@Override
public void mouseExited(MouseEvent e) {}
}
```

Overriding

Overriding the mouseClicked method allows the MonthPanelClickListener class to specify what action should be taken when a mouse click event occurs on a month panel. In this case, it creates and displays a new calendar window.

8- Method of Calendar:

Java

```
// Method to get an array of dates for each day in the current month
```

```

public LocalDate[][] getDaysInMonth() {
    LocalDate[][] result = new LocalDate[5][7]; // Create 2D
    array for dates
    LocalDate firstDayOfMonth = LocalDate.of(year, month + 1, 1); //
    Get first day of the month

    int daysInMonth = firstDayOfMonth.lengthOfMonth(); // Get
    number of days in the month int dayOfWeek =
    firstDayOfMonth.getDayOfWeek().getValue(); // Get day of week
    for the first day
    int day = 1; // Initialize day counter

    // Loop through each cell in the calendar grid
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 7; j++) {
            if ((i == 0 && j < dayOfWeek) || day > daysInMonth) {
                result[i][j] = null; // Set date to null
            } else {
                result[i][j] = LocalDate.of(year, month + 1, day); //
                Set date to corresponding day day++; // Increment day
                counter
            }
        }
    }

    return result; // Return the array of dates
}

```

- `LocalDate[][] result = new LocalDate[5][7];`: This line creates a 2D array to store the dates. It assumes a maximum of 5 weeks (rows) and 7 days (columns) in a month.
- `LocalDate firstDayOfMonth = LocalDate.of(year, month + 1, 1);`: It gets the first day of the current month using the `LocalDate.of()` method. The `month + 1` is used because month is zero-indexed (January is 0, February is 1, and so on), while `LocalDate` uses a one-indexed month.
- `int daysInMonth = firstDayOfMonth.lengthOfMonth();`: This line calculates the total number of days in the current month using the `lengthOfMonth()` method.
- `int dayOfWeek = firstDayOfMonth.getDayOfWeek().getValue();`: It determines the day of

the week for the first day of the month. This value is useful for determining the starting position of the dates in the grid.

- int day = 1;; This initializes a counter to keep track of the day currently being processed.
- Nested loops iterate through each cell in the calendar grid (5 rows and 7 columns), representing each week and each day of the week.

Code:

Java

```
import java.awt.*;
import javax.swing.*;
import java.time.LocalDate;
import java.awt.event.MouseEvent;
import java.awt.event.MouseListener;

public class Year_Calendar_Application extends JFrame {

    private Container container;
    private LocalDate currentDate;
    private JPanel monthGridPanel;

    //constructor

    public Year_Calendar_Application() {

        super("Calendar");
        container = getContentPane();
        container.setLayout(new BorderLayout());

        // Initialize the calendar view with the current date

        currentDate = LocalDate.now();

        // Create and add the label for the year 2024 at the top

        JLabel yearLabel = new JLabel("2024");
        yearLabel.setBorder(BorderFactory.createEmptyBorder(0,
            20, 0, 0)); // Set border container.add(yearLabel,
```

```

        BorderLayout.NORTH);
yearLabel.setFont(new Font("Serif", Font.PLAIN, 40)); // Set
font for the year label

// Create grid panel for months
monthGridPanel = createGridPanel();

// Set the month grid panel as the default content
container.add(monthGridPanel, BorderLayout.CENTER);

// Set frame properties
setSize(800, 600);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLocationRelativeTo(null);
setVisible(true);
}

// Method to create the grid panel for displaying 12 months

private JPanel createGridPanel() {

    JPanel gridPanel = new JPanel();
    gridPanel.setLayout(new GridLayout(3, 4)); // Set layout as
3x4 grid

// Loop through all months to create panels for each

    for (int month = 0; month < 12; month++) {
        JPanel monthPanel = createMonthPanel(month); // Create
panel for the month monthPanel.addMouseListener(new
MonthPanelClickListener(month)); // Add mouse listener
        gridPanel.add(monthPanel); // Add month panel to the grid
    }

    return gridPanel;
}

// Inner class for handling click events on month panels
private class MonthPanelClickListener implements MouseListener
{

```

```

private int month;

public MonthPanelClickListener(int month) {
    this.month = month;
}

// Open the calendar when a grid is clicked
@Override
public void mouseClicked(MouseEvent e) {
    Calendar calendar = new Calendar(); // Create calendar window
    calendar.setVisible(true); // Set calendar window as visible
}

// Other mouse event methods
@Override
public void mousePressed(MouseEvent e) {}
@Override
public void mouseReleased(MouseEvent e) {}
@Override
public void mouseEntered(MouseEvent e) {}
@Override
public void mouseExited(MouseEvent e) {}
}

// Method to create panel for each month

private JPanel createMonthPanel(int month) {
    JPanel monthPanel = new JPanel();
    monthPanel.setLayout(new BorderLayout());
    monthPanel.setBorder(BorderFactory.createLineBorder(Color.BLACK)); // Set border

    // Add label for the month name
    JLabel monthLabel = new JLabel(new String[]{"January",
"February", "March", "April", "May", "June", "July",
    "August", "September", "October", "November",
    "December"}[month],
JLabel.CENTER);
    monthPanel.add(monthLabel, BorderLayout.NORTH); // Add month

```

```

        label to the top

// Create panel for day names
JPanel dayNamesPanel = createDayNamesPanel();
monthPanel.add(dayNamesPanel, BorderLayout.CENTER); // Add
day names panel

// Create panel for days in the month
JPanel daysPanel = createDaysPanel(month);
monthPanel.add(daysPanel, BorderLayout.SOUTH); // Add days
panel to the bottom

return monthPanel;
}

// Method to create panel for displaying day names
private JPanel createDayNamesPanel() {
    JPanel dayNamesPanel = new JPanel(new GridLayout(1, 7)); //
Create panel with 1 row and 7 columns (7 Days of the week)
    String[] dayNames = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat",
"Sun"}; // Array of day names

    // Loop through day names
    for (String dayName : dayNames) {
        JLabel dayLabel = new JLabel(dayName, JLabel.CENTER); // Create
label for day name

        if (dayName.equals("Sun")) { // If it's Sunday, set color to
red
            dayLabel.setForeground(Color.RED);
        }
        dayNamesPanel.add(dayLabel); // Add day label to the panel
    }
    return dayNamesPanel; // Return the panel
}

// Method to create panel for displaying days in a month
private JPanel createDaysPanel(int month) {
    JPanel daysPanel = new JPanel(new GridLayout(0, 7)); //
Create panel with variable rows and 7 columns

```

```

// Calculate the first day of the month
LocalDate firstDayOfMonth =
    LocalDate.of(currentDate.getYear(), month + 1, 1); int
    dayOfWeek = firstDayOfMonth.getDayOfWeek().getValue();
// Get day of week

// Add empty labels for days before the first day of the month
for (int i = 1; i < dayOfWeek; i++) {
    daysPanel.add(new JLabel("")); // Add empty label
}

// Add days to the panel
for (int day = 1; day <= firstDayOfMonth.lengthOfMonth();
    day++) {
    JLabel dayLabel = new JLabel(String.valueOf(day),
JLabel.CENTER); // Create label for day
    daysPanel.add(dayLabel); // Add day label to the panel
    if ((day - 1 + dayOfWeek) % 7 == 0) { // If it's Sunday, set
        color to red
        dayLabel.setForeground(Color.RED);
    }
}

// Calculate the number of empty labels needed at the end of
the month
int totalDays = dayOfWeek + firstDayOfMonth.lengthOfMonth() -
1;
int remainingEmptyLabels = 7 - (totalDays % 7);

// Add remaining empty labels at the end of the month
for (int i = 0; i < remainingEmptyLabels; i++) {
    daysPanel.add(new JLabel("")); // Add empty label
}

return daysPanel; // Return the panel
}

// Main method
public static void main(String[] args) {

```

```

        new Year_Calendar_Application(); // Create an instance of
        Year_Calendar_Application }
    }
    // Calendar class for displaying a detailed calendar view
    class Calendar extends JFrame {

        // Variables for GUI components and calendar data
        private JLabel monthLabel, yearLabel;
        private JButton prevMonthButton, nextMonthButton,
        prevYearButton, nextYearButton; private JPanel
        calendarPanel;
        private JButton[][] dayButtons;
        private int month, year;
        private Container container;
        private LocalDate currentDate;

        // Constructor
        public Calendar() {
            super("Calendar"); // Set the title of the frame
            container = getContentPane(); // Get the content
            pane of the frame container.setLayout(new
            BorderLayout()); // Set layout of the content
            pane

            // Header Panel
            JPanel headerPanel = new JPanel(); // Create header panel
            monthLabel = new JLabel(" ",
            JLabel.CENTER); // Label for month
            yearLabel = new JLabel(" ", JLabel.CENTER);
            // Label for year

            // Create custom icons for navigation buttons
            Icon leftArrowIcon = new
            ImageIcon("C:\\Demo\\right_row.png");
            Icon rightArrowIcon = new
            ImageIcon("C:\\Demo\\left_row.png");

            // Create buttons with custom icons for navigation
            prevMonthButton =
            createIconButton(leftArrowIcon, "previous

```

```

month");          nextMonthButton          =
createIconButton(rightArrowIcon,          "Next
month");          prevYearButton          =
createIconButton(leftArrowIcon,          "previous
Year");          nextYearButton          =
createIconButton(rightArrowIcon, "next Year");

// Add action listeners to the navigation buttons
prevMonthButton.addActionListener(e -> {
    if (month > 0) {
        month--;
    } else {
        month = 11;
        year--;
    }
    updateCalendar();
});
nextMonthButton.addActionListener(e -> {
    if (month < 11) {
        month++;
    } else {
        month = 0;
        year++;
    }
    updateCalendar();
});

prevYearButton.addActionListener(e -> {
    year--;
    updateCalendar();
});

nextYearButton.addActionListener(e -> {
    year++;
    updateCalendar();
});

// Add components to the header panel
headerPanel.add(prevYearButton);
headerPanel.add(prevMonthButton);

```

```

headerPanel.add(monthLabel);
headerPanel.add(yearLabel);
headerPanel.add(nextMonthButton);
headerPanel.add(nextYearButton);

container.add(headerPanel, BorderLayout.NORTH); // Add header
panel to the top

// Calendar Panel
calendarPanel = new JPanel();
calendarPanel.setLayout(new BorderLayout()); // Set layout

// Panel for displaying day names
JPanel dayNamesPanel = new JPanel(new GridLayout(1, 7)); //
Create panel with 1 row and 7 columns(7 days of a Week)
String[] dayNames = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri",
"Sat"}; // Array of day names

// Add day name labels to the panel
for (String dayName : dayNames) {
    JLabel dayLabel = new JLabel(dayName, JLabel.CENTER); //
    Create label for day name dayNamesPanel.add(dayLabel); //
    Add day label to the panel
}

calendarPanel.add(dayNamesPanel, BorderLayout.NORTH); //
Add day names panel to the top

JPanel tablePanel = new JPanel(new GridLayout(0, 7)); //
Create panel for table layout dayButtons = new
JButton[5][7]; // 2D array for storing day buttons

// Loop to create buttons for each day in the calendar

for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 7; j++) {
        dayButtons[i][j] = new JButton(); // Create button
        dayButtons[i][j].setBorderPainted(false); // Remove
        border to make it aesthetic
        dayButtons[i][j].setContentAreaFilled(false); //

```



```

        Make        content        area        transparent
        dayButtons[i][j].setOpaque(true); // Make button
        opaque
        dayButtons[i][j].setBackground(Color.WHITE); // Set
        background color
        tablePanel.add(dayButtons[i][j]); // Add button to the
        table panel
    }
}

calendarPanel.add(tablePanel, BorderLayout.CENTER); // Add

table panel to the center container.add(calendarPanel,

BorderLayout.CENTER); // Add calendar panel to the center

// Set default month and year
currentDate = LocalDate.now(); // Get current date
month = currentDate.getMonthValue() - 1; // Get current month
year = currentDate.getYear(); // Get current year

updateCalendar(); // Update calendar view

setSize(650, 500); // Set size of the frame
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
// Close operation setLocationRelativeTo(null);
// Center the frame
setVisible(true); // Make the frame visible
}

// Method to create a button with custom icon and tooltip

private JButton createIconButton(Icon icon, String tooltip) {
    JButton button = new JButton(icon); // Create button with icon
    button.setToolTipText(tooltip); // Set tooltip text
    button.setBackground(Color.WHITE); // Set background color
    button.setOpaque(true); // Make button opaque
    return button; // Return the button
}

```

```
// Method to update the text and appearance of day buttons in the calendar
```

```
private void updateDayButton(JButton button, LocalDate date) {  
    if (date != null) { // If date is not null  
        button.setText(String.valueOf(date.getDayOfMonth())); //  
        Set button text to day of month  
        button.setForeground(Color.BLACK); // Set text color to black  
  
        // Check if the current button corresponds to the current day  
        LocalDate today = LocalDate.now(); // Get current date  
        if (date.isEqual(today)) { // If date is equal to current date  
            button.setBackground(Color.PINK); // Set background color to pink  
        } else { // If not current date  
            button.setBackground(Color.WHITE); // Set background color to white  
        }  
    } else { // If date is null  
        button.setText(""); // Set button text to empty string  
        button.setBackground(Color.WHITE); // Set background color to white  
    }  
}
```

```
// Method to update the calendar view with the current month and year
```

```
private void updateCalendar() {  
    // Set text for month and year labels  
    monthLabel.setText(new String[]{"January", "February",  
    "March", "April", "May", "June", "July",  
        "August", "September", "October", "November",  
        "December"}[month]);  
    yearLabel.setText(String.valueOf(year));  
  
    // Get array of dates for each day in the current month
```

```

        LocalDate[][] daysInMonth = getDaysInMonth();

        // Update each day button in the calendar panel
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 7; j++) {
                updateDayButton(dayButtons[i][j], daysInMonth[i][j]);
            }
        }
    }

    // Method to get an array of dates for each day in the current month
    public LocalDate[][] getDaysInMonth() {
        LocalDate[][] result = new LocalDate[5][7]; // Create 2D array for dates
        LocalDate firstDayOfMonth = LocalDate.of(year, month + 1, 1); // Get first day of the month

        int daysInMonth = firstDayOfMonth.lengthOfMonth(); // Get number of days in the month
        int dayOfWeek = firstDayOfMonth.getDayOfWeek().getValue(); // Get day of week for the first day

        int day = 1; // Initialize day counter

        // Loop through each cell in the calendar grid
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 7; j++) {
                if ((i == 0 && j < dayOfWeek) || day > daysInMonth) {
                    result[i][j] = null; // Set date to null
                } else {
                    result[i][j] = LocalDate.of(year, month + 1, day); // Set date to corresponding day
                    day++; // Increment day counter
                }
            }
        }

        return result; // Return the array of dates
    }
}

```

```
// Main method
public static void main(String[] args) {
    new Calendar(); // Create an instance of the Calendar class
}
}
```

Additional Source:

<https://stackoverflow.com/questions/35679827/how-to-display-calendar-in-java> (idea overview of the structure of basic structure of code Calendar)

<https://youtu.be/H7kvp-0AX10?feature=shared> (overview of LocalDate)

<https://youtu.be/0XgdX5hDL4U?feature=shared> (overview of LocalDate)

<https://stackoverflow.com/questions/17232038/calendar-display-using-java-swing> (idea overview of the structure of calendar)